

CLARITAS: Explainable AI Misinformation & Trust Detection Engine

Intra IIT Mid Prep Technical Documentation & System Architecture Report

Team Members:

Anvay Joshi (CS23B060) Raghavendra Pappu (CS23B036) Srikrishna Kidambi (CS23B058)

*Department of Computer Science and Engineering
Indian Institute of Technology*

September 29, 2026

Abstract

The proliferation of digital misinformation across online news portals and social platforms presents a critical threat to public trust and decision-making. Standard black-box deep learning classifiers often suffer from opaque decision boundaries, poor domain generalization, and vulnerability to subtle linguistic style shifts. In this paper and project submission, we introduce **Claritas**, an end-to-end, privacy-focused Misinformation & Trust Detection Platform. Claritas leverages a novel **Two-Layer Stacking Ensemble Architecture** combining a fine-tuned DistilRoBERTa transformer (Layer 1) with a calibrated LightGBM meta-classifier (Layer 2) trained on stylistic, sentiment, readability, and source prior feature vectors. Furthermore, Claritas integrates a privacy-preserving **Local Explainable AI (XAI) engine** powered by Ollama/SHAP/LIME, live web content extraction, line-by-line batch pipeline execution, and a persistent Human-In-The-Loop (HITL) audit queue. On a held-out test dataset of 8,980 samples, Claritas achieves an accuracy of **99.93%**, precision of **100.00%**, recall of **99.87%**, and an AUC-ROC of **0.99999**.

Contents

1	Introduction & Problem Statement	3
1.1	Problem Statement	3
1.2	Objectives of Claritas	3
2	System Architecture & Pipeline Design	3
2.1	Module 1: Live Web Content Scraper	4
2.2	Module 2: Feature Engineering	4
3	Two-Layer Stacking Ensemble Classification	5
3.1	Layer 1: Deep Fine-Tuned DistilRoBERTa	5
3.2	Layer 2: Calibrated LightGBM Meta-Classifier	5
3.3	Classification Decision Boundaries	5
4	Experimental Results & Model Evaluation	5
4.1	Performance Metrics Table	5
4.2	Confusion Matrix	6

5	Explainable AI (XAI) & Privacy-Focused Local LLM Integration	6
5.1	1. SHAP Feature Attribution	6
5.2	2. LIME Token Highlighting	6
5.3	3. Local Ollama LLM Explanation Summaries	6
6	User Interface & Software Architecture	6
7	Installation, Setup & Reproducibility Guide	7
7.1	Directory Structure	7
7.2	Backend Setup Instructions	7
7.3	Frontend Setup Instructions	8
8	Conclusion & Future Enhancements	8
8.1	Summary of Contributions	8
8.2	Future Work	8

1 Introduction & Problem Statement

1.1 Problem Statement

In modern digital media consumption, false claims, automated propaganda, and sensationalist clickbait spread rapidly before fact-checkers can perform manual verification. Automated misinformation detectors face three major challenges:

1. **The Black-Box Dilemma:** High-performing neural models (e.g., Transformers) provide raw probabilities without explaining *why* a claim was flagged, limiting user trust.
2. **Style vs. Semantic Drift:** Models trained solely on semantics fail when misinformation uses formal news formatting, or when real news contains emotional rhetoric.
3. **Privacy & Operational Overhead:** Offloading explanation generation to cloud LLMs introduces data privacy risks, API cost bottlenecks, and external service dependencies.

1.2 Objectives of Claritas

Claritas is engineered to address these challenges by delivering:

- **Layered Ensemble Classification:** Combining deep stylistic representations with explicit linguistic metrics (polarity, subjectivity, reading ease, casing ratios).
- **Local Explainable AI (XAI):** Delivering SHAP feature attribution bars, inline token highlighting (LIME), and plain-English natural language summaries via locally hosted LLMs without cloud API calls.
- **Live Web Scraping & Batch Processing:** Enabling instant URL content extraction and line-by-line batch claim verification.
- **Human-In-The-Loop (HITL) Audit System:** Providing a persistent audit queue to log auditor overrides and update domain credibility tracking dynamically.

2 System Architecture & Pipeline Design

The Claritas pipeline consists of four tightly integrated modules: Data Ingestion & Web Extraction, Feature Engineering, Two-Layer Classification, and Explainable AI / HITL Audit.

High-Level Data Flow Architecture

[User Input: Text / URL / Batch]

↓

[Module 1: Web Scraper & Content Cleaner] (BeautifulSoup4 + Requests)

↓

[Module 2: Feature Extraction Pipeline]

└ Layer 1: DistilRoBERTa Semantic & Style Probability (P_{RoBERTa})

└ Linguistic Metrics: Polarity, Subjectivity, VADER Compound, Flesch Reading Ease, Caps Ratio

└ Domain Prior: Historical Domain Credibility Score (S_{domain})

↓

[Module 3: Layer 2 Calibrated LightGBM Meta-Classifier] → Final Verdict & Confidence

↓

[Module 4: Explainability & HITL Audit Engine]

└ SHAP Feature Attribution & LIME Token Highlighting

└ Local Ollama LLM Summary Generation

└ Human Review Queue Persistence

Figure 1: Claritas End-to-End System Pipeline.

2.1 Module 1: Live Web Content Scraper

When a user provides a web URL (e.g., `https://...`), the scraper extracts article content while stripping noise:

1. Fetches page HTML using spoofed browser headers to bypass basic user-agent blocks.
2. Decomposes clutter tags: `<script>`, `<style>`, `<nav>`, `<header>`, `<footer>`, `<aside>`, `<iframe>`.
3. Extracts paragraphs (`<p>`) with character length > 20 . If paragraph length < 150 , falls back to structural body extraction.
4. Triggers a `ScrapingError` if client-side JavaScript rendering blocks plain text extraction.

2.2 Module 2: Feature Engineering

For any cleaned article text T , Claritas computes a 10-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{10}$:

Feature Name	Symbol	Description / Metric Definition
RoBERTa Probability	P_{RoBERTa}	Layer 1 transformer probability score $\in [0, 1]$
Sentiment Polarity	S_{polarity}	TextBlob polarity score $\in [-1.0, 1.0]$
Subjectivity Index	S_{subj}	TextBlob subjectivity score $\in [0.0, 1.0]$
VADER Compound	V_{compound}	NLTK VADER sentiment intensity score $\in [-1.0, 1.0]$
Flesch Reading Ease	F_{flesch}	Readability score (0–100 scale)
Average Word Length	W_{avglen}	Mean character count per word
Capitalization Ratio	R_{caps}	Proportion of uppercase characters in text
Exclamation Ratio	R_{excl}	Frequency of exclamation marks relative to total length
Word Count	N_{words}	Total token count
Source Domain Score	S_{domain}	Bayesian historical credibility prior of domain $\in [0, 1]$

Table 1: 10-Dimensional Feature Vector used by Layer 2 Meta-Classifer.

3 Two-Layer Stacking Ensemble Classification

3.1 Layer 1: Deep Fine-Tuned DistilRoBERTa

Layer 1 uses a fine-tuned `distilroberta-base` transformer. The input text sequence is tokenized and passed through transformer encoder layers:

$$\mathbf{h}_{\text{CLS}} = \text{DistilRoBERTa}(T) \quad (1)$$

The classification head outputs logits z_0, z_1 , transformed into probability P_{RoBERTa} via Softmax:

$$P_{\text{RoBERTa}} = \frac{\exp(z_1)}{\exp(z_0) + \exp(z_1)} \quad (2)$$

3.2 Layer 2: Calibrated LightGBM Meta-Classifier

The probability P_{RoBERTa} is concatenated with the 9 engineered linguistic and source metrics to construct feature vector $\mathbf{x}$. A LightGBM gradient-boosted decision tree ensemble outputs raw decision margins, which are passed through Sigmoid Isotonic Calibration:

$$P_{\text{final}} = \mathcal{C}_{\text{isotonic}}(\text{LightGBM}(\mathbf{x})) \quad (3)$$

3.3 Classification Decision Boundaries

The final verdict and confidence score are assigned using tuned decision thresholds:

$$\text{Verdict} = \begin{cases} \text{Likely Real,} & \text{if } P_{\text{final}} \leq 0.40 \quad (\text{Confidence} = (1 - P_{\text{final}}) \times 100\%) \\ \text{Likely Misinformation,} & \text{if } P_{\text{final}} \geq 0.60 \quad (\text{Confidence} = P_{\text{final}} \times 100\%) \\ \text{Uncertain,} & \text{if } 0.40 < P_{\text{final}} < 0.60 \quad (\text{Confidence} = (1 - 2|P_{\text{final}} - 0.5|) \times 100\%) \end{cases} \quad (4)$$

4 Experimental Results & Model Evaluation

Claritas was evaluated on a held-out test split of $N = 8,980$ sample news claims.

4.1 Performance Metrics Table

Evaluation Metric	Held-Out Test Performance
Accuracy	99.93% (0.9993)
Precision	100.00% (1.0000)
Recall	99.87% (0.9987)
F1-Score	99.94% (0.9994)
AUC-ROC	0.99999

Table 2: Quantitative Performance on Held-Out Test Dataset ($N = 8,980$).

4.2 Confusion Matrix

		Predicted Class	
		Predicted Real	Predicted Fake
2*Actual Class	Actual Real	4,284 (True Real)	0 (False Positive)
	Actual Fake	6 (False Negative)	4,690 (True Fake)

Table 3: Confusion Matrix breakdown ($N = 8,980$). Zero False Positives ensures no genuine news articles were falsely flagged as fake.

5 Explainable AI (XAI) & Privacy-Focused Local LLM Integration

Claritas provides three levels of explainability:

5.1 1. SHAP Feature Attribution

Tree SHAP computes exact Shapley values ϕ_i for each metric x_i in Layer 2:

$$\phi_i(\mathbf{x}) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (5)$$

The relative weight percentage of each metric is rendered in the UI as signal strength indicators.

5.2 2. LIME Token Highlighting

Local Interpretable Model-agnostic Explanations (LIME) perturbs token sequences to compute individual word weights:

$$w_t = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (6)$$

Tokens pushing toward misinformation are highlighted in red, while tokens supporting factual credibility are highlighted in green.

5.3 3. Local Ollama LLM Explanation Summaries

To preserve user privacy and avoid cloud API costs, Claritas connects locally to an **Ollama** instance running Llama 3. It formats metric contributions and LIME weights into a structured prompt, returning plain-English bullet points explaining the decision.

6 User Interface & Software Architecture

The frontend is built with React.js (Vite) and Tailwind CSS, following a component-driven architecture:

Frontend Component Hierarchy

```

App.jsx (React Router)
├─ Navbar.jsx (Sticky navigation header)
├─ AnalysisContext.jsx (Global Context provider preserving state across tab switches)
├─ AnalyzerPage.jsx (Main single/batch input, extracted URL text collapse preview, SHAP/LIME charts)
├─ VerdictCard.jsx (Verdict badge, confidence percentage, LIME token display)
├─ EvidencePanel.jsx (SHAP breakdown interactive LIME trigger)
├─ HumanReviewModal.jsx (HITL audit override modal)
├─ ReviewQueuePage.jsx (Filterable/sortable audit history table)
├─ HowItWorksPage.jsx (System metrics, confusion matrix, interactive architecture diagram)

```

Figure 2: React Component Architecture.

7 Installation, Setup & Reproducibility Guide

7.1 Directory Structure

```

claritas/
data/
  domain_history.json
src/
  backend/
    models/ (DistilRoBERTa & LightGBM weights)
    classifier.py
    main.py
    metrics.py
    scraper.py
  components/
    AnalyzerPage.jsx
    HowItWorksPage.jsx
    HumanReviewModal.jsx
    ReviewQueuePage.jsx
    VerdictCard.jsx
  App.jsx
  main.jsx
package.json
vite.config.js

```

7.2 Backend Setup Instructions

```

# 1. Navigate to backend directory
cd claritas/src/backend

# 2. Create and activate Python virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install required Python packages
pip install fastapi uvicorn torch transformers lightgbm pandas \

```

```
textblob textstat nltk beautifulsoup4 requests joblib shap lime
# 4. Start FastAPI backend server
python main.py
```

The FastAPI service starts at `http://localhost:8000`.

7.3 Frontend Setup Instructions

```
# 1. Navigate to claritas project root
cd claritas

# 2. Install NPM dependencies
npm install

# 3. Launch Vite development server
npm run dev
```

The React application starts at `http://localhost:5173`.

8 Conclusion & Future Enhancements

8.1 Summary of Contributions

In this project, we successfully engineered **Claritas**, a complete end-to-end Explainable AI Misinformation Detection Engine. Key achievements include:

- Achieving **99.93% accuracy** and **100.00% precision** on held-out test data using a two-layer stacking ensemble.
- Solving the black-box dilemma via local SHAP attribution, LIME token highlights, and local Ollama LLM summaries.
- Eliminating state resets during live web scraping by isolating persistence files outside the Vite HMR directory.
- Providing full batch processing and persistent Human-In-The-Loop audit capabilities.

8.2 Future Work

- **Multimodal Verification:** Extending Layer 1 to accept synthetic image detection features alongside text.
- **Active Learning Auto-Tuning:** Automatically re-triggering LightGBM calibration steps as human audit feedback accumulates in the review queue.

— End of Documentation —